

Scope, Side Effects, and Function Design

Day 4 Morning - Functions and Program Design

30 min teaching + 30 min exercises

```
def calculate_tax(price, tax_rate):  
    return price * tax_rate  
  
tax = calculate_tax(50, 0.07)
```

```
def normalize_record(record):  
    record["status"] =  
    record["status"].strip().lower()
```

```
normalize_record()  
validate_record()  
calculate_metrics()  
format_record_summary()
```

Goal: write functions that are clear, explicit, reusable, and easier to test.

BLOCK 27

Where this block fits

Block 26 focused on returned values. Block 27 focuses on function boundaries.



Instruction focus

- local and global variables
- parameters as local names
- side effects and pure functions
- single responsibility
- separating calculation from display

Practice focus

Students refactor repeated logic into small functions with explicit inputs and limited surprises.

The central question

When a function runs, which names can it see, and what does it change?

Scope

Where a variable name is available.

Side effect

A change a function makes outside its returned value.

Design question

Can a reader understand this function by looking only at its name, parameters, and return value?

BLOCK 27

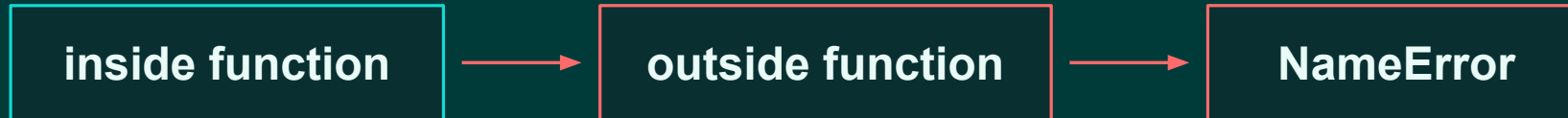
Local variables live inside the function

A variable created inside a function normally exists only there.

```
def calculate_total(price, quantity):  
    total = price * quantity  
    return total  
  
print(total)
```

What happens?

The name `total` is local to `calculate_total()`.
Outside the function, Python cannot see it.



A local variable is temporary workspace for that function call.

BLOCK 27

Parameters are local names too

Arguments come from the caller; parameter names exist inside the function call.

```
def double_value(number):  
    result = number * 2  
    return result  
  
answer = double_value(5)
```

argument value
5



parameter name
number

Mental model

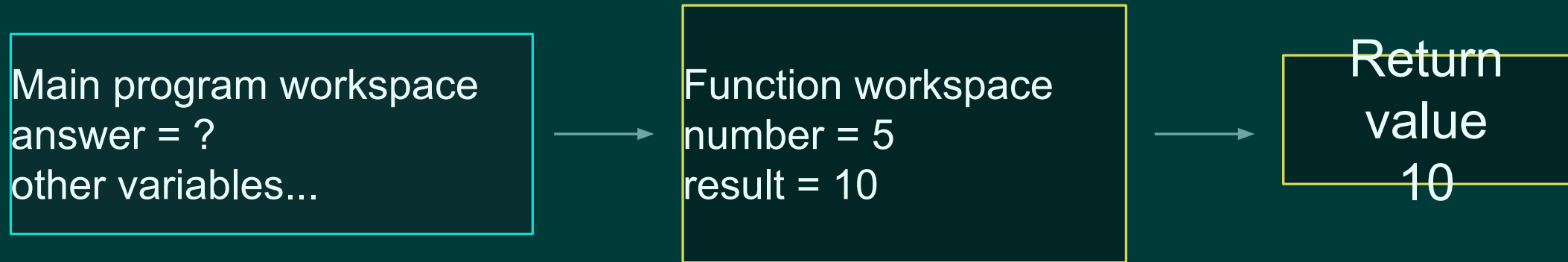
When the function starts, number is a local name that currently holds 5.

The caller does not need a variable named number. The function creates that local name.

BLOCK 27

A function call gets its own small workspace

Think of local scope as a temporary workbench.



Key idea

When the call finishes, the local workspace is gone. Only the returned value remains available to the caller.

BLOCK 27

Global variables live outside functions

A function can read a global, but this can create hidden dependencies.

```
tax_rate = 0.07

def calculate_tax(price):
    return price * tax_rate

print(calculate_tax(50))
```

What the signature says
calculate_tax(price)

Looks like it only needs price.

What it secretly needs
tax_rate from elsewhere in the file.

This is not always wrong, but it is easier to misunderstand and harder to reuse.

BLOCK 27

Make dependencies explicit

Pass required information into the function instead of hiding it elsewhere.

```
def calculate_tax(price):  
    return price * tax_rate
```



```
def calculate_tax(price, tax_rate):  
    return price * tax_rate
```

Hidden input
Depends on `tax_rate` outside the
function.

Explicit input
All required information appears in the
parameter list.

Practical rule: prefer passing information in when it is part of the calculation.

Why excessive globals hurt

Global state makes programs more sensitive to where and when code runs.

Harder to understand
Function behavior depends
on variables far away.

Harder to test
A test must also prepare
hidden external state.

Harder to reuse
Moving the function may
break its hidden assumptions.

Order matters more
A global may have been
changed before the function
runs.

Accidents are easier
One function can change
something another function
expected.

Globals are not forbidden. Hidden, unnecessary dependencies are the real problem.

Variable lifetime

Most local names exist only while that function call is running.



```
def calculate():  
    result = 100  
    return result  
  
answer = calculate()  
print(answer)
```

Safe pattern
Return the value, then store it in a
name in the caller.

BLOCK 27

Side effects: changing something outside the return value

Functions can modify mutable objects such as lists and dictionaries.

```
def normalize_record(record):  
    record["status"] =  
record["status"].strip().lower()  
  
record = {"status": " ACTIVE "  
normalize_record(record)  
print(record)
```

What changed?

The original dictionary was modified.

Is this always bad?

No. Sometimes this is intentional. But it should not be surprising.

Side effects matter because the caller may not expect its data to be changed.

BLOCK 27

Side effect versus returned value

Both approaches can work; the important part is choosing intentionally.

```
def normalize_record(record):  
    record["status"] =  
record["status"].strip().lower()
```

Modifies existing dictionary
The caller keeps the same object, but its contents have changed.

```
def normalize_status(status):  
    return status.strip().lower()
```

Returns a new value
The caller decides whether to store or ignore the result.

Beginner habit: return values unless your function's job is specifically to update an object.

BLOCK 27

Pure functions are easier to reason about

A pure function calculates from inputs without changing unrelated program state.

```
def calculate_percentage(completed, total):  
    return completed / total * 100
```

Inputs
18, 24



Function



75.0

Same input, same output

Pure functions are usually easier to understand, reuse, test, and debug.

BLOCK 27

Single responsibility

A function should have one clear job.

```
def process_everything(record):  
    # normalize strings  
    # validate fields  
    # calculate metrics  
    # print messages  
    # update counters  
    # save results
```

Problem

The function name does not reveal which job failed or which part can be reused.

Better question

Can this function be explained in one sentence without using “and”?

Large functions often hide several smaller functions trying to get out.

BLOCK 27

Split the work into named responsibilities

The main program should read like a high-level explanation.



```
for record in records:
    cleaned = normalize_record(record)
    status, message = validate_record(cleaned)
    metrics = calculate_metrics(cleaned)
    print_record_summary(cleaned, status, message, metrics)
```

BLOCK 27

Function names should be verbs

A good name tells the reader what action the function performs.

Prefer

`calculate_percentage()`

`normalize_status()`

`validate_record()`

`classify_record()`

`create_summary()`

Avoid

`do_it()`

`handle_data()`

`process_stuff()`

`thing()`

`run_logic()`

Naming test

Could another student guess what this function returns just by reading its name and parameters?

BLOCK 27

Separate calculation from display

Calculation functions should usually return; display functions should print.

```
def calculate_percentage(completed, total):  
    percentage = completed / total * 100  
    print(f"{percentage:.1f}%")
```

Less reusable

The calculation is trapped inside screen output.

```
def calculate_percentage(completed, total):  
    return completed / total * 100
```

```
percentage = calculate_percentage(18, 24)  
print(f"{percentage:.1f}%")
```

More reusable

The caller decides whether to print, store, compare, or test.

Separate “figuring out” from “telling the human.”

BLOCK 27

Design checklist for this morning

Use this checklist when writing or refactoring functions.

Inputs are explicit

Does the parameter list show what the function needs?

Output is clear

Does the return value have an obvious meaning?

One job

Can the function be explained in one sentence?

Few surprises

Does it avoid unnecessary globals and hidden side effects?

Testable

Could you call it with sample inputs and check the result?

BLOCK 27

Guided exercise setup

Refactor messy code into explicit, reusable functions.

Practice rule

Do not start by writing new code. First identify the responsibilities already hidden in the script.

Time plan

Exercise 1: 10 min

Exercise 2: 5 min

Exercise 3: 7 min

Exercise 4: 5 min

Challenge: remaining time

The goal is not shorter code. The goal is code with clearer boundaries.

BLOCK 27

Exercise 1: refactor repeated tax logic

Turn repeated calculations into small functions.

```
tax_rate = 0.07

price = 50
quantity = 4
subtotal = price * quantity
tax = subtotal * tax_rate
total = subtotal + tax
print(total)

price = 25
quantity = 7
subtotal = price * quantity
tax = subtotal * tax_rate
total = subtotal + tax
print(total)
```

Task

Identify repeated operations.
Create functions for subtotal, tax, and total.

10 minutes

Rule

Pass `tax_rate` into the function.
Do not depend on the global name.

BLOCK 27

Exercise 1 target structure

Use clear functions, then call them for each purchase.

```
def calculate_subtotal(price, quantity):  
    return price * quantity  
  
def calculate_tax(subtotal, tax_rate):  
    return subtotal * tax_rate  
  
def calculate_total(subtotal, tax):  
    return subtotal + tax
```

```
subtotal = calculate_subtotal(50, 4)  
tax = calculate_tax(subtotal, 0.07)  
total = calculate_total(subtotal, tax)  
print(total)
```

Checkpoint

Can each function be tested independently with one print statement?

BLOCK 27

Exercise 2: remove the hidden global

Make the function show every value it depends on.

```
maximum_errors = 5

def is_valid_error_count(error_count):
    return error_count <= maximum_errors
```

Task

Rewrite the function so `maximum_errors` is passed in as a parameter.

5 minutes

Discussion

When might a constant near the top of a file be acceptable?
When is it hiding important behavior?

BLOCK 27

Exercise 3: spot the side effect

Predict what prints, then rewrite to avoid surprising changes.

```
def normalize_record(record):  
    record["status"] =  
record["status"].strip().lower()  
  
record = {  
    "record_id": "EQ-1001",  
    "status": " ACTIVE ",  
}  
  
normalize_record(record)  
print(record)
```

Task

1. Predict the output.
2. Explain what changed.
3. Return a cleaned dictionary.

7 minutes

Stretch: decide which version is better for a data-processing pipeline.

BLOCK 27

Exercise 4: split responsibilities

Identify functions inside a Day 3-style processing loop.

```
for record in records:
    record_id =
record["record_id"].strip().upper()
    status = record["status"].strip().lower()

    if record_id == "":
        print("Rejected: missing ID")
    elif record["error_count"] > 5:
        print("Rejected: too many errors")
    elif status == "active":
        print("Ready")
    else:
        print("Review")
```

Task

Mark which lines belong to:
normalization, validation, classification,
and output formatting.

5 minutes

Do not implement yet
The first skill is recognizing the hidden
jobs.

BLOCK 27

Applied challenge: design the target pipeline

Convert the large loop into named function calls.

```
for record in records:

    cleaned_record = normalize_record(record)

    status, message =
validate_record(cleaned_record)

    metrics = calculate_metrics(cleaned_record)

    print_record_summary(
        cleaned_record,
        status,
        message,
        metrics
    )
```

Challenge

Sketch what each function should receive and return.

Implement one function at a time.

Success criteria

The loop explains what happens without showing every low-level detail.

Remaining time

BLOCK 27

Block 27 wrap-up

Students should now see functions as boundaries, not just reusable snippets.

Students can explain

- local scope
- parameters as local names
- global variables
- side effects
- pure functions

Students can do

- pass dependencies explicitly
- avoid surprising state changes
- split large functions
- name functions clearly
- separate calculation from display

Next block: documentation, type hints, and composing functions into a workflow.